sheLLaMa User Manual

AI-Powered Agentic Shell for Bash & PowerShell

Version 1.0 — May 2026

Contents

1 Introduction

sheLLaMa is an AI-powered agentic shell that integrates directly into bash and PowerShell. The AI reads files, writes code, runs commands, searches the web, and iterates—with permission tiers, conversation memory, and context awareness.

1.1 Key Capabilities

- Chat with AI using conversation memory
- Agentic mode: AI proposes and executes commands iteratively (up to 10 rounds)
- Structured file tools: read, write, and diff-based edit
- Web search for documentation and error messages
- Image/vision analysis with multimodal models
- Code generation, explanation, and multi-file analysis
- Shell-to-Ansible playbook conversion
- Text-to-image generation (Stable Diffusion)
- Conversation save/load, branching, and auto-compaction
- Fully offline after initial model pull

2 Installation

2.1 Prerequisites

- Python 3.8+ (for bash CLI)
- PowerShell 5.1+ (for PowerShell clients)
- Access to a sheLLaMa backend server (or run your own)

2.2 Bash CLI (Linux/macOS)

```
git clone https://github.com/pfworks/sheLLaMa.git
cd shellama
export SHELLAMA_API=http://your-server:5000
./cli/shellama
```

2.3 Bash Integration (Recommended)

Add to your `.bashrc`:

```
source /path/to/shellama/cli/shellama.bash
```

This gives you all , commands in your real bash session with full job control, history, tab completion, and aliases.

2.4 PowerShell CLI (Windows)

```
$env:SHELLAMA_API = "http://your-server:5000"
.\powershell\powershellama.ps1
```

2.5 PowerShell Integration (Recommended)

Add to your `$PROFILE`:

```
. C:\path\to\shellama\powershell\shellama.ps1
```

3 Commands Reference

All commands are prefixed with `,` (comma). Regular shell commands work normally.

3.1 Chat & Agentic Mode

Command	Description
`, <prompt>`	Chat with AI (conversation memory, no command execution)
`,do <prompt>`	Agentic mode—AI runs commands, asks `Run? [y/N/q]`
`,, <prompt>`	Quiet mode—output only, no confirmations

3.2 Code & File Operations

Command	Description
`,explain <file>`	Explain any file (auto-detects playbook vs code)
`,generate <desc>`	Generate code or Ansible playbook from description
`,analyze <paths>`	Analyze files and/or directories recursively
`,save <file>`	Save last AI output to a file (strips markdown fences)

3.3 Image & Vision

Command	Description
`,img <prompt>`	Generate image from text (Stable Diffusion)
`,vision <image> [prompt]`	Send image to AI for analysis (requires multimodal model)

3.4 Session Management

Command	Description
`,session save [name]`	Save conversation to `~/.shellama/sessions/`
`,session load [name]`	Load a saved session (interactive picker or by name)
`,session list`	List all saved sessions

3.5 Context Files

Context files are attached to every prompt automatically, giving the AI persistent awareness of your project.

Command	Description
`,context add <file>`	Attach file to every prompt
`,context remove <file>`	Remove file from context
`,context list`	Show attached files with sizes
`,context clear`	Remove all context files

3.6 Conversation Branching

Command	Description
`,branch`	Fork conversation to explore a tangent
`,branch back`	Return to main conversation (discards branch)

Branching is stack-based—you can nest multiple branches.

3.7 Utilities

Command	Description
`,models`	List and select model
`,test [model|all]`	Benchmark models (speed, tokens, cloud cost)
`,tokens`	Show session usage stats
`,quiet`	Toggle quiet mode
`,list` / `,help`	Show available commands
`,exit`	Unload sheLLaMa (integration mode only)

4 AI Tools (Agentic Mode)

When you use `,do` or `,` (in the standalone CLI), the AI has access to five structured tools:

4.1 bash / powershell

Runs shell commands. Permission tiers apply:

- **Auto-allowed**: read-only commands (ls, cat, grep, git status, curl, etc.)
- **Prompted**: everything else—asks `Run? [y/N/q]`
- **Blocked**: destructive commands (rm -rf /, mkfs, git push –force, etc.)—always refused

4.2 file_read

Reads files and returns numbered lines. Auto-allowed (no confirmation needed). Safer than `cat` because it handles binary detection and truncation.

4.3 file_write

Creates new files or replaces entire file content. Shows a preview of the first 5 lines and asks `Write? [y/N/q]`.

4.4 file_edit

Targeted search-and-replace in existing files using the format:

```
<<<< SEARCH
exact lines to find
====
replacement lines
>>>> END
```

Multiple edits can be applied in one block. Shows a diff preview and asks `Apply? [y/N/q]`. Reports which edits matched and which failed.

4.5 web_search

Searches DuckDuckGo for documentation, APIs, and error messages. Auto-allowed. Returns top 5 results with titles and snippets.

5 Auto-Compaction

When a conversation exceeds 24,000 characters (~6,000 tokens), older turns are automatically summarized by the LLM. The system prompt and last 2 exchanges are kept intact. A message `[compacted: 28000 → 12000 chars]` confirms when this happens.

Configure the threshold:

```
export SHELLAMA_COMPACT_THRESHOLD=32000  # higher = more context before
    compacting
```

6 Configuration

6.1 Environment Variables

Variable	Default	Description
SHELLAMA_API	http://192.168.1.229:5000	API endpoint
SHELLAMA_MODEL	auto	Default model
SHELLAMA_API_KEY	(empty)	API key for authentication
AI_IMAGE_MODEL	sdxl-turbo	Image generation model
SHELLAMA_COMPACT_THRESHOLD	24000	Auto-compact at this char count
SHELLAMA_DOWNLOAD_DIR	(current dir)	Save directory for images
AI_QUIET	false	Start in quiet mode
AI_PS1	(bash PS1)	Custom prompt (bash CLI only)

6.2 Recommended Models

Model	Response Time	Notes
qwen2.5-coder:3b	10–30s	Fast, decent quality

`qwen2.5-coder:7b`	30–60s	Best balance
`deepseek-coder:6.7b`	30–60s	Alternative
`qwen2.5-coder:14b`	1–3min	Higher quality, needs 32+ cores
`llava:7b`	30–60s	Multimodal (for `,vision`)

7 Tips & Best Practices

1. **Use context files** for project awareness: `,context add README.md`
2. **Use `,branch`** before trying risky approaches—return with `,branch back`
3. **Save sessions** before closing terminal: `,session save myproject`
4. **Use `,do`** for tasks that need commands; use `,` for questions
5. **The AI prefers file_edit** over file_write for existing files—this is safer
6. **Check `,tokens`** to monitor usage during long sessions

8 Troubleshooting

"nothing to save (no AI output yet)" — Run an AI command first, then `,save`.

Slow responses — Use a smaller model (`3b` or `7b`). Check backend load.

"cannot reach" error — Verify `SHELLAMA_API` is set correctly and the server is running.

Permission denied on commands — Blocked commands cannot be overridden from the CLI. Ask your admin to add patterns to the allow list via the Settings page.

Context too large — Remove large files with `,context remove`. The AI shows total bytes injected per prompt.